9. Level 4: Code-level diagrams

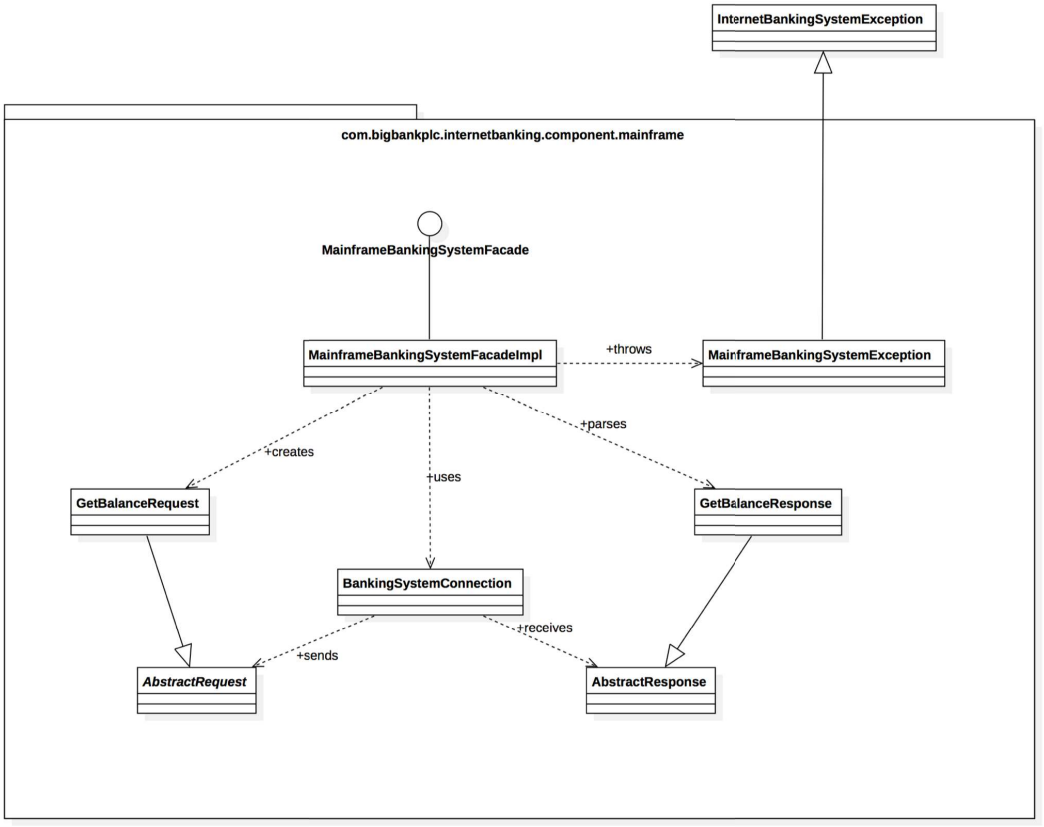
The final level of detail is one or more code-level diagrams showing the implementation details of an individual component.

9.1 Intent

The intent of a code-level diagram is to illustrate the structure of the code and, in this case, how a component is implemented.

9.2 Structure

If you're using an object-oriented programming language, probably the best way to do this is to use a UML class diagram, either by generating it automatically from the code or by drawing it freehand. Let's look at an example by zooming in on the "Mainframe Banking System Facade" component from the Internet Banking System "API Application" container.



A UML class diagram

How much detail?

The danger with UML class diagrams is that it's very easy to include a considerable amount of detail, and this is especially true if you are auto-generating diagrams from code. Although it's tempting to include every field/property/attribute and method, I would resist this temptation and only include as much information as you need to tell the story that you want to tell. Typically, I will only include the attributes and methods that are relevant to the narrative I want to create.

UML class diagrams have often been used to describe an entire application, but this just results in a huge mess of overlapping boxes and lines, regardless of how well-structured the code is. The key to using class diagrams is to limit their scope. In this case, scope is limited to the internals of a component.

9.3 Motivation

Having this final level of abstraction provides a way to map the high-level, coarse-grained components into real-world code elements. It helps to bridge what are sometimes seen as two very different worlds; the software architecture and the code.

9.4 Audience

Technical people within the software development team, specifically software developers.

9.5 Required or optional?

Since this level of detail lives in the code, software developers can get this detail *on demand* from the code itself. Therefore, this is definitely an optional level of detail, and generally not recommended. I don't typically draw class diagrams for anything but the most complex of components, or as a template when I want to describe a pattern that is used across a codebase.